



Department of Artificial Intelligence and Machine Learning

ARTIFICIAL NEURAL NETWORKS AND DEEP LEARNING

Course Code : AI253IA

Date : 29/1/2025

Semester : V Sem

Time : 2:30pm to 4:30pm

Max Marks : 10 (Q) + 50(CIE)

Duration : 30 + 90=120 min

Re-Test

Note: Answer all the Questions

Q. No		Questions	M	BT	CO																
1	a)	What is striding in CNN?	2	2	1																
	b)	Examine the effectiveness of the ReLU layer compared to other activation functions (sigmoid and Tanh) in CNNs.	2	1	1																
	c)	How would you analyse the application of Recurrent Neural Networks (RNNs) in automatic image captioning, and what role do they play in generating textual descriptions of images?	2	3	1																
	d)	Evaluate the challenges of Reinforcement Learning in real-world applications like self-driving cars and conversational systems, and suggest possible solutions.	2	2	1																
	e)	Give the max pooling considering 2×2 window size	2	2	1																
		<table><tr><td>6</td><td>8</td><td>3</td><td>2</td></tr><tr><td>1</td><td>7</td><td>5</td><td>8</td></tr><tr><td>5</td><td>6</td><td>4</td><td>5</td></tr><tr><td>3</td><td>2</td><td>8</td><td>5</td></tr></table>				6	8	3	2	1	7	5	8	5	6	4	5	3	2	8	5
6		8				3	2														
1		7				5	8														
5		6				4	5														
3	2	8	5																		

Re-Test

Note: Answer all the Questions

Q. No	Questions	M	B T	C O
2	a) Examine the architectures of Gated Recurrent Unit (GRU) and Long Short-Term Memory (LSTM) models, derive the output equations for each layer, and analyse the distinctions between the two.	5	3	1
	b) Explain Back propagation, through matrix multiplication in Convolutional Neural Network (CNN)	5	2	1
3	a) Demonstrate the process of backpropagation through convolutions with an example matrix calculation: (Considering: Input Image (3x3 matrix): Input = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] Filter (Kernel) (2x2 matrix): Filter = [[1, 0], [0, -1]]). Compute the following: a. Output feature map b. Backpropagation to update the weights: Assume the loss function L has been calculated and its gradient concerning the output is: $\partial L / \partial \text{Output} = [[1, 1], [1, 1]]$ c. Update the filter weights	10	3	3
	b) What is data augmentation? Explain with example	5	2	1

4	a)	Consider the convolutional neural network defined by the layers in the left column below. Fill in the shape of the output volume and the number of parameters at each layer. You can write the shapes in the NumPy format (e.g. (64,64,3)). Notation: 1. CONV5-N denotes a convolutional layer with N filters with height and width equal to 5. Padding is 2, and stride is 1. 2. POOL2 denotes a 2x2 max-pooling layer with stride of 2 and 0 padding. 3. FC-N denotes a fully connected layer with N neurons.	1 0	3	3																					
		<table><tr><td>Layer</td><td>Activation Volume Dimensions</td><td>Number of parameters</td></tr><tr><td>Input</td><td>32*32*1</td><td>0</td></tr><tr><td>CONV5-10</td><td></td><td></td></tr><tr><td>POOL2</td><td></td><td></td></tr><tr><td>CONV5-10</td><td></td><td></td></tr><tr><td>POOL2</td><td></td><td></td></tr><tr><td>FC10</td><td></td><td></td></tr></table>				Layer	Activation Volume Dimensions	Number of parameters	Input	32*32*1	0	CONV5-10			POOL2			CONV5-10			POOL2			FC10		
		Layer				Activation Volume Dimensions	Number of parameters																			
		Input				32*32*1	0																			
		CONV5-10																								
		POOL2																								
		CONV5-10																								
		POOL2																								
FC10																										
5	a)	How does Backpropagation Through Time (BPTT) differ from regular backpropagation in feed-forward networks, and what are the challenges faced when applying it to RNNs, particularly in terms of gradient explosion or vanishing gradients?	5	3	2																					
	b)	Explain any two approaches of CNN for video classification ?	5	4	3																					
6		Compare the performance of basic RNNs, LSTMs, and GRUs in handling long-term dependencies in time-series forecasting. What are the strengths and weaknesses of each model	5	3	2																					

M-Marks, BT-Blooms Taxonomy Levels, CO-Course Outcomes

Marks Distribution	Particulars	C01	C02	C03	C04	L1	L2	L3	L4	L5	L6
	Max Marks CIE & Quiz	30	5	15	-	4	12	20	14	10	-

Course Outcomes

C01:	Describe basic concepts of neural networks, its applications and various learning models
C02:	Analyse different network architectures, learning tasks, CNN, and deep learning models
C03:	Investigate and apply neural networks model and learning techniques to solve problems related to society and industry.
C04:	Demonstrate a prototype application developed using any NN tools and APIs.
C05:	Appraise the knowledge of neural networks and deep learning as an individual/as a team member.

Old Test - Quiz Solutions

Date: 29th January 2025

1. What is striding in CNN?

Striding refers to the number of pixels the kernel (filter) moves across the input matrix during the convolution operation. It is a hyperparameter that controls the step size of the convolution.

- **Stride = 1:** The filter moves one pixel at a time. This results in an output that is roughly the same spatial dimension as the input (depending on padding).
- **Stride > 1:** The filter jumps over pixels (e.g., a stride of 2 means the filter moves 2 pixels at a time). This results in **downsampling**, effectively reducing the spatial dimensions (height and width) of the output feature map.

Mathematical Impact: If an input has size W , filter size is K , padding is P , and stride is S , the output size O is calculated as:

$$O = \lfloor \frac{W - K + 2P}{S} \rfloor + 1$$

2. Examine the effectiveness of the ReLU layer compared to other activation functions (sigmoid and tanH) in CNNs.

The **Rectified Linear Unit (ReLU)**, defined as $f(x) = \max(0, x)$, has become the standard activation function in Convolutional Neural Networks (CNNs), largely replacing Sigmoid and Tanh for hidden layers.

Comparison:

Feature	ReLU	Sigmoid / Tanh
Vanishing Gradient	Effective: ReLU does not saturate for positive values. The gradient is either 0 or 1, allowing errors to backpropagate effectively through deep layers.	Prone: Both functions saturate at their tails (values near 0 or 1 for Sigmoid; -1 or 1 for Tanh). This causes gradients to approach zero, halting learning in deep networks.
Computational Cost	Low: It requires simple thresholding (if $x > 0$, output x ; else 0). No expensive exponential calculations.	High: Both involve computing exponential functions (e^x), which is computationally expensive for large datasets.
Sparsity	High: ReLU outputs true zero for negative inputs, leading to sparse representations (fewer active neurons), which can simplify the model and reduce overfitting.	Low: These functions almost always produce non-zero dense outputs, meaning all neurons are firing to some degree.

Conclusion: ReLU is generally more effective for CNNs because it accelerates convergence and enables the training of much deeper networks without the vanishing gradient problem.

3. How would you analyze the application of Recurrent Neural Networks in automatic image captioning, and what role do they play in generating textual descriptions of images?

Analysis of Application: Automatic image captioning typically employs an **Encoder-Decoder architecture**.

1. **Encoder (CNN):** A pre-trained CNN (like ResNet or VGG) is used to extract a rich feature vector from the input image. This vector encapsulates the visual content.
2. **Decoder (RNN/LSTM):** This feature vector is passed to the Recurrent Neural Network.

Role of RNNs in Generation: The RNN acts as the language model component of the system. Its specific roles include:

- **Sequential Processing:** Unlike feedforward networks, RNNs have "memory" (hidden states) that allows them to process sequences. Language is inherently sequential (word order matters).
- **Conditioned Generation:** The RNN is initialized with the image features. At each time step, it predicts the probability of the next word in the sentence based on:
 1. The previously generated word.
 2. Its current internal state (context).
 3. The original image features (often enhanced by Attention mechanisms).
- **End-to-End Dependency:** It ensures the generated sentence is grammatically correct and semantically coherent, continuing to generate words until it predicts a special <END> token.

4. Evaluate the challenges of Reinforcement Learning in real-world applications like self-driving cars and conversational systems, and suggest possible solutions.

Challenges:

- **Self-Driving Cars (Safety & Sample Efficiency):**
 - **Exploration vs. Safety:** In RL, agents often learn by failing. A self-driving car cannot "explore" by crashing into a wall to learn it is bad.
 - **Sample Inefficiency:** Deep RL requires millions of episodes to converge. Driving millions of miles in the real world is expensive and slow.
- **Conversational Systems (Reward Modeling):**
 - **Defining Rewards:** It is difficult to mathematically define a "good conversation." Metrics like "time spent" might encourage stalling rather than helping.
 - **Delayed Rewards:** The success of a dialogue often isn't known until the very end, making credit assignment difficult.

Suggested Solutions:

1. **Sim-to-Real Transfer:** Train agents in high-fidelity physics simulators (e.g., CARLA for driving) and then transfer the learned policy to the real world using domain randomization to handle discrepancies.
2. **Inverse Reinforcement Learning (IRL):** Instead of manually designing a reward function, the agent learns the reward function by observing expert human behavior (e.g., watching a human drive or reading human chat logs).
3. **Safe RL / Constrained MDPs:** embed hard constraints into the optimization process so that the agent maximizes reward without violating safety rules (e.g., "never cross the lane divider").
4. **Reinforcement Learning from Human Feedback (RLHF):** For chatbots, humans rank the agent's responses. This data trains a reward model, which then trains the RL agent, bypassing the need for an algorithmic reward function.

5. Max pooling, Average Pooling, explain different types of pooling with examples.

Pooling is a downsampling operation used to reduce the dimensionality of the feature map, decreasing computational load and controlling overfitting.

A. Max Pooling

Selects the **maximum** value from the portion of the image covered by the kernel. It works well to preserve the most prominent features (like bright pixels or strong edges) and provides translation invariance.

B. Average Pooling

Calculates the **average** of all values in the kernel window. It smooths out the image and retains background information better than Max Pooling.

Example Calculation

Consider a 4×4 Input Matrix and a 2×2 Filter with Stride 2.

Input Matrix:

$$\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 8 & 2 & 1 & 0 \\ 3 & 4 & 5 & 2 \end{bmatrix}$$

1. Max Pooling Output:

- Top-Left block $\{1, 3, 5, 6\} \rightarrow$ Max is **6**
- Top-Right block $\{2, 4, 1, 2\} \rightarrow$ Max is **4**
- Bottom-Left block $\{8, 2, 3, 4\} \rightarrow$ Max is **8**
- Bottom-Right block $\{1, 0, 5, 2\} \rightarrow$ Max is **5**

Result:

$$\begin{bmatrix} 6 & 4 \\ 8 & 5 \end{bmatrix}$$

2. Average Pooling Output:

- Top-Left block $\{1, 3, 5, 6\} \rightarrow \text{Sum } 15 / 4 = \mathbf{3.75}$
- Top-Right block $\{2, 4, 1, 2\} \rightarrow \text{Sum } 9 / 4 = \mathbf{2.25}$
- Bottom-Left block $\{8, 2, 3, 4\} \rightarrow \text{Sum } 17 / 4 = \mathbf{4.25}$
- Bottom-Right block $\{1, 0, 5, 2\} \rightarrow \text{Sum } 8 / 4 = \mathbf{2.0}$

Result:

$$\begin{bmatrix} 3.75 & 2.25 \\ 4.25 & 2.0 \end{bmatrix}$$

6. Examine the architectures of GRU and LSTM models, derive the output equations for each layer and analyze the distinctions between the two.

LSTM Architecture & Equations: Long Short-Term Memory (LSTM) networks are designed to avoid the vanishing gradient problem. They utilize three gates to control information flow.

- **Forget Gate (f_t):** Decides what information to discard from the cell state.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- **Input Gate (i_t):** Decides which values to update in the cell state.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

Candidate cell state: $\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$

- **Cell State Update (C_t):** Combines the old state and the new candidate.

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

- **Output Gate (o_t) & Hidden State (h_t):** Determines the next hidden state.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

GRU Architecture & Equations: Gated Recurrent Units (GRU) simplify the LSTM architecture by combining gates.

- **Update Gate (z_t):** Decides how much of the past information needs to be passed along to the future.

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

- **Reset Gate (r_t):** Decides how much of the past information to forget.

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

- **Current Memory Content ($\tilde{h}_t$):**

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

- **Final Memory (h_t):**

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Distinctions:

1. **State Structure:** LSTM maintains two separate states: the cell state (C_t) for long-term memory and the hidden state (h_t) for output. GRU combines these into a single hidden state (h_t).
2. **Complexity:** LSTM has three gates (Input, Output, Forget), whereas GRU has only two (Update, Reset). This makes GRU computationally more efficient and faster to train.
3. **Data Flow:** In LSTM, the amount of memory content exposed is controlled by the output gate. In GRU, the full memory content is exposed at every step.

7. Explain Back Propagation, through matrix multiplication in CNN

In Convolutional Neural Networks (CNNs), backpropagation is used to calculate the gradients of the loss function with respect to the weights (filters) and the inputs, allowing the network to learn.

1. Forward Pass (Convolution as Matrix Multiplication): While convolution is a sliding window operation, it can be represented as a matrix multiplication $Y = X \cdot W$ by converting the input X into a **Toeplitz matrix** (often using a method called `im2col`) and flattening the filter W . This allows the use of optimized linear algebra libraries.

2. Backward Pass (Gradients): Assume we have the error gradient from the next layer, δ^{l+1} (often denoted as $\frac{\partial L}{\partial Y}$). We need to compute two things:

- **Gradient w.r.t. Filter (Weights):** We need to find how changes in the filter weights affect the loss. This is calculated as the convolution of the **Input (X)** and the **Error (δ^{l+1})**.

$$\frac{\partial L}{\partial W} = X * \delta^{l+1}$$

Note: In matrix terms, this operation resembles the cross-correlation of the input feature map and the error map.

- **Gradient w.r.t. Input (Passing error to previous layer):** To propagate the error back to the previous layer l , we calculate the gradient with respect to the input X . This is mathematically equivalent to the "full" convolution of the **Error** (δ^{l+1}) and the **Rotated Filter** (W_{rot180}).

$$\delta^l = \frac{\partial L}{\partial X} = \delta^{l+1} * W_{rot180}$$

Note: The filter must be rotated 180 degrees to mathematically satisfy the chain rule for convolution operations during backpropagation.

8. Demonstrate the process of back propagation through convolutions with an example matrix calculation

Given:

- **Input Image (X):** A 3×3 matrix (Note: provided input data is 3×3).

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

- **Filter / Kernel (W):** A 2×2 matrix.

$$W = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

- **Loss Gradient ($\frac{\partial L}{\partial Y}$):** The gradient of the loss function with respect to the output map.

$$\frac{\partial L}{\partial Y} = \delta_{output} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

a) Compute Output Feature Map (Y)

We perform a valid correlation (stride=1). $Y_{i,j} = \sum_m \sum_n X_{i+m,j+n} \cdot W_{m,n}$

- **Top-Left ($Y_{0,0}$):** $(1 \cdot 1) + (2 \cdot 0) + (4 \cdot 0) + (5 \cdot -1) = 1 - 5 = -4$
- **Top-Right ($Y_{0,1}$):** $(2 \cdot 1) + (3 \cdot 0) + (5 \cdot 0) + (6 \cdot -1) = 2 - 6 = -4$
- **Bottom-Left ($Y_{1,0}$):** $(4 \cdot 1) + (5 \cdot 0) + (7 \cdot 0) + (8 \cdot -1) = 4 - 8 = -4$
- **Bottom-Right ($Y_{1,1}$):** $(5 \cdot 1) + (6 \cdot 0) + (8 \cdot 0) + (9 \cdot -1) = 5 - 9 = -4$

$$\text{Output } Y = \begin{bmatrix} -4 & -4 \\ -4 & -4 \end{bmatrix}$$

b) Back Propagation to find Weight Gradients ($\frac{\partial L}{\partial W}$)

To update the weights, we need to find how much each weight contributed to the error. This is calculated by convolving the **Input** (X) with the **Output Gradient** (δ_{output}).

$$\frac{\partial L}{\partial W_{m,n}} = \sum_i \sum_j \delta_{output_{i,j}} \cdot X_{i+m,j+n}$$

1. For $W_{0,0}$ (Top-Left Weight): Multiply gradient matrix by the input patch corresponding to this weight's position in the sliding window. $= (1 \cdot 1) + (1 \cdot 2) + (1 \cdot 4) + (1 \cdot 5)$
 $= 1 + 2 + 4 + 5 = \mathbf{12}$

2. For $W_{0,1}$ (Top-Right Weight): Shift input focus one step right.
 $= (1 \cdot 2) + (1 \cdot 3) + (1 \cdot 5) + (1 \cdot 6) = 2 + 3 + 5 + 6 = \mathbf{16}$

3. For $W_{1,0}$ (Bottom-Left Weight): Shift input focus one step down.
 $= (1 \cdot 4) + (1 \cdot 5) + (1 \cdot 7) + (1 \cdot 8) = 4 + 5 + 7 + 8 = \mathbf{24}$

4. For $W_{1,1}$ (Bottom-Right Weight): Shift input focus down and right.
 $= (1 \cdot 5) + (1 \cdot 6) + (1 \cdot 8) + (1 \cdot 9) = 5 + 6 + 8 + 9 = \mathbf{28}$

Resulting Weight Gradient:

$$\frac{\partial L}{\partial W} = \begin{bmatrix} 12 & 16 \\ 24 & 28 \end{bmatrix}$$

c) Update the filter weights

The weights are updated using the learning rate (η). New Weights = Old Weights - (Learning Rate $\times$ Gradient)

$$W_{new} = W - \eta \cdot \frac{\partial L}{\partial W}$$

$$W_{new} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} - \eta \begin{bmatrix} 12 & 16 \\ 24 & 28 \end{bmatrix}$$

$$W_{new} = \begin{bmatrix} 1 - 12\eta & -16\eta \\ -24\eta & -1 - 28\eta \end{bmatrix}$$

9. Explain the CNN Concepts: how object localization is effectively performed in an image, highlighting the key techniques, layers, and processes involved.

Object Localization differs from simple Image Classification. While classification asks "What is in the image?", localization asks "What is in the image, and **where** is it?" (typically by drawing a bounding box around a single object).

The Architecture (Multi-Task Learning)

Localization is usually treated as a **regression problem** alongside classification. The architecture typically involves a shared "backbone" and two separate "heads."

1. The Backbone (Feature Extractor): A standard CNN (e.g., VGG, ResNet) takes the raw input image and processes it through convolutional and pooling layers. This extracts high-level feature maps that contain information about the object's appearance and position.

2. The Classification Head:

- **Layer:** Fully Connected (Dense) Layer + Softmax.
- **Function:** Outputs a probability distribution over class labels (e.g., Dog: 0.8, Cat: 0.2).
- **Loss:** Cross-Entropy Loss (L_{cls}).

3. The Regression (Localization) Head:

- **Layer:** Fully Connected (Dense) Layer with linear activation (or sometimes sigmoid if normalized).
- **Function:** Outputs 4 continuous numbers defining the bounding box: (b_x, b_y, b_w, b_h) .
 - b_x, b_y : Coordinates of the box center.
 - b_w, b_h : Width and height of the box.
- **Loss:** L2 (Mean Squared Error) or Smooth L1 Loss (L_{loc}).

Key Techniques & Processes

1. Defining the Bounding Box: The network needs a ground truth to learn from. Bounding boxes are defined by coordinates relative to the image size.

- $(x_{min}, y_{min}, x_{max}, y_{max})$: Top-left and bottom-right corners.
- $(x_{center}, y_{center}, width, height)$: Center point and dimensions (more common in regression).

2. Intersection over Union (IoU): IoU is the metric used to evaluate how accurate a predicted bounding box is compared to the ground truth.

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

- **IoU = 1:** Perfect match.
- **IoU = 0:** No overlap.
- Generally, an $IoU > 0.5$ is considered a "correct" detection.

3. Anchor Boxes (for advanced localization/detection): In more complex systems (like YOLO or SSD), the network does not predict raw coordinates from scratch. Instead, it predicts "offsets" or adjustments to pre-defined boxes called **Anchor Boxes** (or priors) of different aspect ratios and scales placed across the image.

4. The Loss Function (Joint Training): To train the network for both tasks simultaneously, we use a weighted sum of the two losses:

$$\text{Total Loss} = L_{cls} + \lambda L_{loc}$$

Where λ is a hyperparameter balancing the importance of accurate location versus accurate classification.

Summary of Process:

- 1. **Input:** Image with a labeled object (Class + Bounding Box).
- 2. **Forward Pass:** Image $\rightarrow$ CNN $\rightarrow$ Feature Vector $\rightarrow$ (Head 1: Class Probabilities, Head 2: Box Coordinates).
- 3. **Loss Calculation:** Compute Cross-Entropy for class and MSE for box coordinates.
- 4. **Backpropagation:** Update weights in the backbone and both heads to minimize the total loss.

10. Consider the convolution neural network defined by the layers in the left column below. Fill in the shape of the output volume and number of parameters at each layer.

Notation:

- **CONV5-N:** Convolutional layer with N filters, kernel size 5×5 , padding 2, stride 1.
- **POOL2:** Max pooling layer with 2×2 kernel, stride 2, padding 0.
- **FC-N:** Fully connected layer with N neurons.

Solution Table:

Layer	Activation Volume Dimensions	Number of Parameters
Input	(32, 32, 1)	0
CONV5-10	(32, 32, 10)	260
POOL2	(16, 16, 10)	0
CONV5-10	(16, 16, 10)	2510
POOL2	(8, 8, 10)	0
FC-10	(10, 1)	6410

Detailed Calculations:

- 1. **CONV5-10 (Layer 1):**
 - **Input:** $32 \times 32 \times 1$
 - **Dims:** $H_{out} = \frac{32-5+2(2)}{1} + 1 = 32$. Volume = (32, 32, 10)

- **Params:** $((5 \times 5 \times 1) + 1_{bias}) \times 10 = (25 + 1) \times 10 = \mathbf{260}$

2. POOL2 (Layer 2):

- **Input:** $32 \times 32 \times 10$
- **Dims:** $H_{out} = \frac{32-2}{2} + 1 = 16$. Volume = $(16, 16, 10)$
- **Params:** Pooling has **0** parameters.

3. CONV5-10 (Layer 3):

- **Input:** $16 \times 16 \times 10$
- **Dims:** $H_{out} = \frac{16-5+2(2)}{1} + 1 = 16$. Volume = $(16, 16, 10)$
- **Params:** $((5 \times 5 \times 10_{depth}) + 1_{bias}) \times 10 = (250 + 1) \times 10 = \mathbf{2510}$

4. POOL2 (Layer 4):

- **Input:** $16 \times 16 \times 10$
- **Dims:** $H_{out} = \frac{16-2}{2} + 1 = 8$. Volume = $(8, 8, 10)$
- **Params:** **0**

5. FC-10 (Layer 5):

- **Input:** Flattened $(8, 8, 10) \rightarrow 640$ units.
- **Dims:** 10 neurons. Output shape $(10, 1)$.
- **Params:** $(640_{weights} + 1_{bias}) \times 10 = 641 \times 10 = \mathbf{6410}$

11. How does Backpropagation Through Time (BPTT) differ from regular backpropagation, and what are the challenges faced when applying it to RNNs?

Differences from Regular Backpropagation: In a standard feed-forward network, the data flows in one direction (layer $i \rightarrow i + 1$), and gradients flow back directly. However, Recurrent Neural Networks (RNNs) process sequences where the output at time t depends on $t - 1$.

To apply backpropagation, we **unroll** the RNN over time.

1. **Unrolling:** The recurrent loops are stretched out into a long feed-forward network with one layer per time step.
2. **Shared Weights:** Unlike standard networks where each layer has unique weights, in BPTT, the weight matrices (W_{xh} , W_{hh} , W_{hy}) are **shared** across all time steps.
3. **Gradient Accumulation:** When calculating gradients, the error at time T propagates back through $T - 1, T - 2, \dots, 0$. The total gradient for a weight matrix is the sum of the gradients calculated at each time step.

Challenges (Gradient Instability): Because the gradients are multiplied repeatedly (once for each time step) by the recurrent weight matrix W_{hh} , two major issues arise:

- **Vanishing Gradients:** If the largest eigenvalue of $W_{hh} < 1$, the gradients decay exponentially as they go back in time ($0.9^{100} \approx 0$).

- **Result:** The network stops learning from early inputs, effectively giving it "short-term memory" and making it unable to learn long-term dependencies.
- **Exploding Gradients:** If the largest eigenvalue of $W_{hh} > 1$, the gradients grow exponentially ($1.1^{100} \approx 13780$).
 - **Result:** The weights update by massive amounts, causing the loss to oscillate or diverge to NaN (Not a Number), effectively breaking the model.
 - **Solution:** Gradient Clipping (capping the gradient norm).

12. Explain any 2 approaches of CNN for video classification.

Video classification involves analyzing temporal data (motion) in addition to spatial data (appearance).

Approach 1: 3D Convolutional Neural Networks (3D CNNs / C3D)

- **Concept:** Standard CNNs use 2D filters that move over width and height. 3D CNNs use **3D kernels** ($Height \times Width \times Time$) that slide over the spatial dimensions *and* the temporal axis (frames).
- **Mechanism:** This allows the network to capture motion information directly within the early layers. For example, a filter might activate only when a vertical edge moves to the right over 3 frames.
- **Pros:** Captures spatio-temporal features simultaneously.
- **Cons:** Computationally very expensive and creates a massive number of parameters.

Approach 2: Two-Stream Networks

- **Concept:** Decomposes video into two separate components processed by two distinct 2D CNNs.
 1. **Spatial Stream:** Takes a single frame (RGB) as input. It recognizes objects and scenes (e.g., "This is a baseball field").
 2. **Temporal Stream:** Takes a stack of **Optical Flow** frames as input. Optical flow represents the movement of pixels between frames. It recognizes motion patterns (e.g., "Something is swinging rapidly").
- **Fusion:** The predictions from both streams are fused (averaged or concatenated) at the end to make a final classification.
- **Pros:** Often achieves higher accuracy by explicitly modeling motion.

13. Compare the performance of basic RNNs, LSTMs, and GRUs in handling long-term dependencies in time series forecasting.

Feature	Basic RNN	LSTM (Long Short-Term Memory)	GRU (Gated Recurrent Unit)
---------	-----------	-------------------------------	----------------------------

Handling Long-Term Dependencies	Poor. Due to the vanishing gradient problem, basic RNNs cannot recall information from many time steps ago. They are biased towards the most recent inputs.	Excellent. The "Cell State" acts as a superhighway for gradients, allowing information to flow unchanged over long sequences. The Forget Gate explicitly learns what to keep/discard.	Good to Excellent. Like LSTM, it uses gating mechanisms to bypass the vanishing gradient problem. It generally matches LSTM performance on most tasks.
Model Complexity	Low. Simple architecture with a single tanh layer. Fast to train but hard to converge on complex tasks.	High. Contains 3 gates (Input, Forget, Output) and a separate cell state. More parameters to learn.	Medium. Simplified version of LSTM with only 2 gates (Reset, Update) and no separate cell state.
Training Speed	Fastest (per epoch), but may require more epochs or fail to	Slowest. The complex gating operations require more matrix multiplications	Faster than LSTM. With fewer gates and parameters, it converges